EventHub

A design for a campus event discovery and ticketing application

Axel Bouwhuis (s2717026)
Wouter Deen (s3032132)
Xanti Lizanzu (s2829541)

For the course of
Service Oriented Architecture Web Services

UNIVERSITY OF TWENTE.

Universiteit Twente
Enschede, Nederland
April 17, 2026

1 Introduction

1.1 Motivation and Problem Statement

Universities manage a high volume of diverse events across multiple independent stakeholders, ranging from student associations and festival companies to the municipality. Currently, the lack of a centralized promotional medium results in fragmented information silos, which hinders event discovery for the student body and complicates institutional oversight for administrators. The absence of a unified place to aggregate and discover events makes it difficult to coordinate large-scale campus activities or assess the total impact of scheduled events.

1.2 Project Objectives

This project introduces Eventhub, a platform designed to mitigate these inefficiencies through a centralized architectural approach. The primary objective of Eventhub is to provide a scalable solution that integrates the entire event lifecycle, enabling organizers to manage logistics such as ticketing, overseers to maintain administrative oversight, and visitors to easily discover and book events through a single, interoperable service.

2 Architecture

2.1 Service Definition using Domain-Driven Design

Currently, the problem domain for event discovery at the University of Twente in Enschede is characterized by fragmentation and inefficiency. Events are scattered across multiple platforms, such as the university website, the municipality website, and various student association channels (e.g., websites, social media). These operate in isolated bounded contexts with their own rules, data structures, and ubiquitous language. This lack of a unified system creates a disjointed experience for students and organizers, as there is no single source of truth for event information. The core domain here is event aggregation and discovery, where the primary challenge is consolidating and standardizing event data from diverse sources while preserving the unique requirements and terminology of each subdomain (e.g., academic events, social gatherings, or association activities). From a Domain-Driven Design (DDD) perspective, the objective of this application is to address the problem domain by decomposing it into clearly defined subdomains, each mapped to a bounded context and ultimately implemented as microservices. The core domains are event management and ticketing, where the primary business logic lies in creating, updating, and managing events, as well as handling ticket booking, issuance, and validation.

Thus, the objective of the application we developed, is to provide a platform for university events, enabling users to discover, book, and manage tickets for various events. To achieve this, we employ Domain-Driven Design (DDD) principles to decompose the problem domain into manageable subdomains, which are subsequently mapped to bounded contexts and ultimately to microservices. In the context of Domain-Driven Design, subdomains are identified as distinct areas of the business with their own logic and terminology. For this application, the subdomains are:

- **Event management:** core logic for creating, updating, and managing university events.
- **Ticketing:** core logic for ticket booking, issuance, validation, and inventory management.
- **User management:** supporting logic for user authentication, profiles, roles (such as student, organizer, admin), and permissions.
- **Payment processing:** supporting logic for handling payments and refunds.
- **Notifications:** supporting logic for updating users by sending event updates, reminders, and confirmations.

Each subdomain is then mapped to a bounded context, which defines a clear boundary for models, language, and logic. These bounded contexts correspond to specific business capabilities, as can be seen above. Each bounded context is implemented as a microservice, with clear APIs and ownership:

- **Event service:** responsible for creating, updating, and canceling events, managing event details, categories, and schedules.
- **Ticketing service:** handles booking, issuing, validating, and canceling tickets, as well as managing inventory.
- **User service:** authenticates users, manages profiles, roles, and permissions.
- **Payment service:** processes payments, handles refunds, and generates receipts.
- **Notification service:** sends emails and SMS for alerts, reminders, and confirmations.

2.2 Enterprise Architecture

2.2.1 Containerization and Deployment

We developed an ArchiMate enterprise architecture to depict our architecture, especially in the application and technology layer (Figure 1 or enlarged in Figure A.1). The mapping of Docker and Kubernetes concepts to ArchiMate concepts is loosely inspired[1] by the community posts by Jean-Paul Carmona [1] and Sergey Startcev [2]. The microservices will be containerized using Docker, with each service encapsulated in its own container. This ensures consistency across different environments and simplifies the deployment process. Kubernetes will be used for orchestration, managing the deployment, scaling, and operations of the application containers. Istio will be used as a service mesh, and will inject sidecar containers of Envoy, a service proxy, into each pod to facilitate communication within the service mesh. We will use the Istio Ingress Gateway for Kubernetes to manage ingress and expose the microservices externally, since Istio Ingress Gateway works well with Kubernetes Ingress resources. This setup provides a single entry point for client requests, simplifying the system architecture and enhancing security.

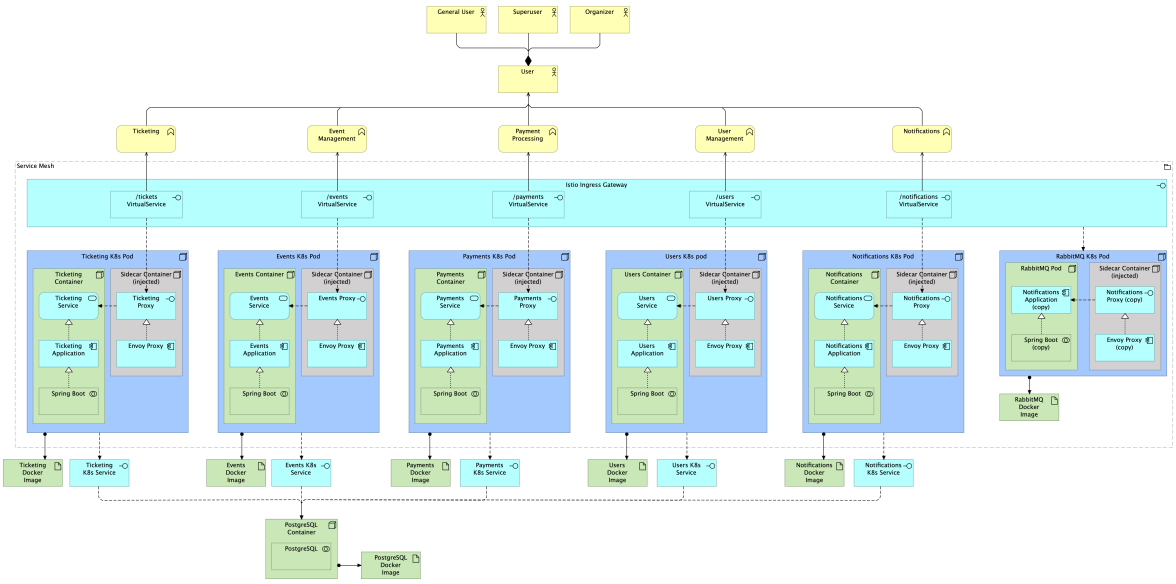

Figure 1: Enterprise architecture of EventHub.

The deployment process will involve creating Kubernetes Deployment and Service configurations for each microservice. These configurations will define the container images, environment variables, and resource requirements for each service. Our microservices will be encapsulated in Kubernetes Pods, and Kubernetes Services will be used to expose the microservices internally. A representation of the different interactions within the architecture can be found in Figure 2.

[1] No design science studies that we could find have been conducted on creating a formal definition for mapping K8s concepts to ArchiMate, hence making this an interesting topic for further research.

The notifications service is our main asynchronous service, using RabbitMQ for queueing notification messages to be sent. For example, the notifications service sends updates to events to the notifications service such that it can send notifications to subscribed users. The payments service also uses a message queue for queueing success payment messages consumed by the notification service. Besides this, most other inter-service communication is via synchronous HTTP REST requests.

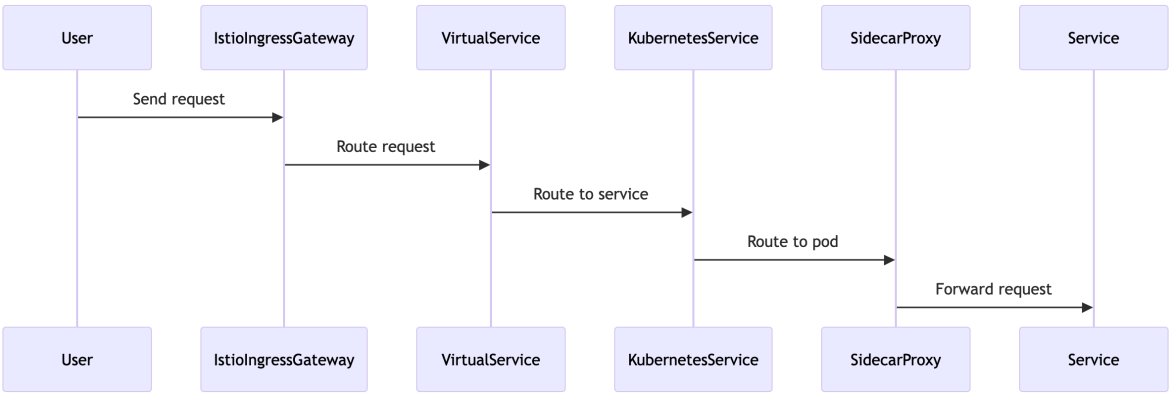

Figure 2: Sequence diagram metamodel of request routing between the different application components.

2.2.2 Database Pattern

We will deploy a single PostgreSQL database outside the cluster as a separate Docker container for persistent data storage. We will use a *schema-per-service pattern*, since this has much lower overhead than a *database-server-per-service pattern*, while having clearer ownership than the *private-tables-per-service pattern* [3]. We can safely assume that there will not be many millions of tickets, therefore, this seems to be the best option when weighing loose coupling with development experience (DevEx) and performance. Additionally, in case this application will scale to other campuses in the Netherlands and the world, the *schema-per-service pattern* allows for easy migration to a database-per-service pattern. Additionally, these databases could be deployed as a StatefulSet.

3 Feasability Analysis

The MVP requires the architecture to be scalable in a dynamic manner. Meaning that certain parts of the service may need extra resources at certain times such as ticketing, payments and notifications. The opening of tickets sales for a popular event may attract thousands of users in a span of minutes. The influx on traffic on certain parts of the services should not limit other functionalities of the service i.e. user and event functionalities.

Furthermore, if we used a monolithic application and wanted to allocate more resources to ticketing and payment, we would also be forced to allocate more resources to user and event functionalities. In the case of a containerized application, resources can be discriminately scaled for each part of the service. This aligns with the SOA principles of service autonomy and loose coupling.

The architecture has a single point of access to the Istio gateway which through its propagated via the Kubernetes cluster. This makes sure that pods and therefore containers cannot be reached directly from the outside, making the system more secure. The architecture is based on a Kubernetes cluster which can easily be adapted and run on a server of a university as part of their domain. This makes the system easy to adapt and deploy for universities.

4 BPMN Models

Two business processes were modeled in Business Process Model Notation (BPMN) to show the interaction of the user at the frontend and the services in the backend. Both show complex interactions

including CRUD operations, message publication and subscriptions between services and processes within services. The models can be found in Figure 3 and Figure 4.

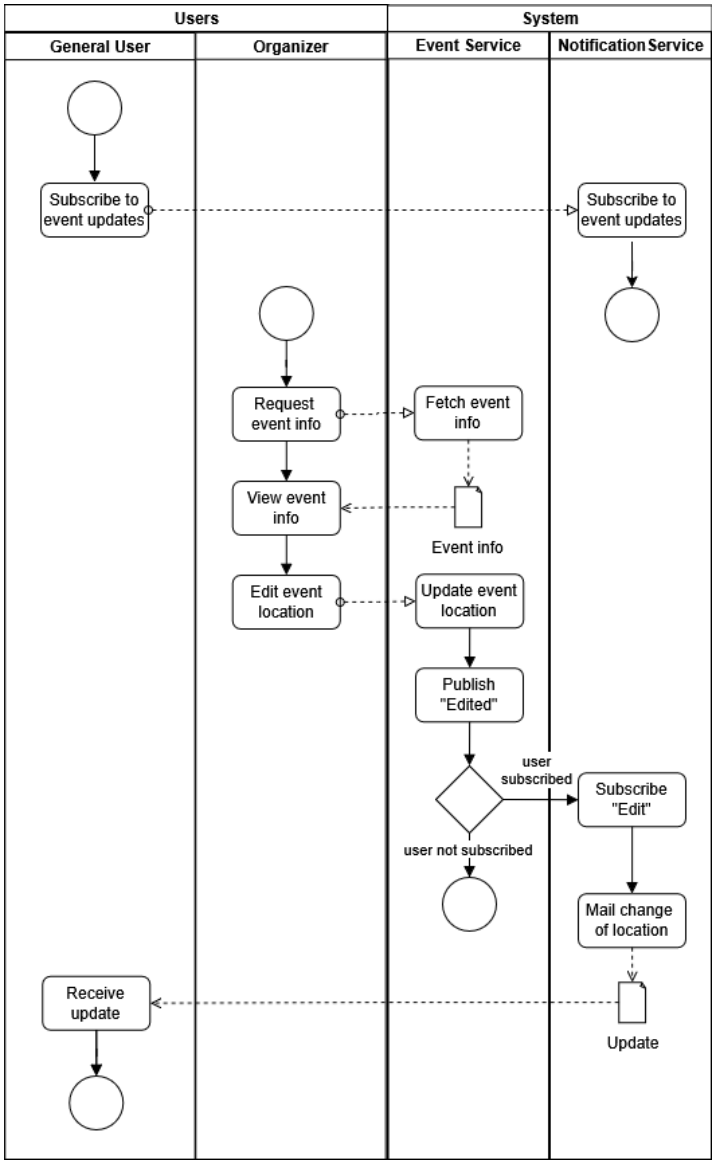

Figure 3: BPMN model of editing the location of an event.

The first business process modeled is the editing of a location of an event. The first step of the process is that an Organizer views an existing event in its frontend. There, the location of the event is edited. The event is updated in the database, as well as that an "Edited" message is published to a message queue. A message queue was chosen since the notification service can figure out itself whether it is ready to tackle sending out all of the mails or that it takes the message from the queue later on. The General User in the end receives the update of the event by mail.

The second business process is buying a ticket. First all events and then the single event of interest are read by simple read operations between the General User and Event Service. For the single event, the Ticket Service is queried for the ticket availability. A ticket is then reserved for 10 minutes, on a time-out the ticket is made available again. If the user chooses to pay and it fails, the same goes. However, if a payment succeeds, the successful payment of a ticket is published as a message in the Payment Service, which the Ticket Service consumes to mark the ticket as bought and the Notification Service consumes to send out a mail with the tickets.

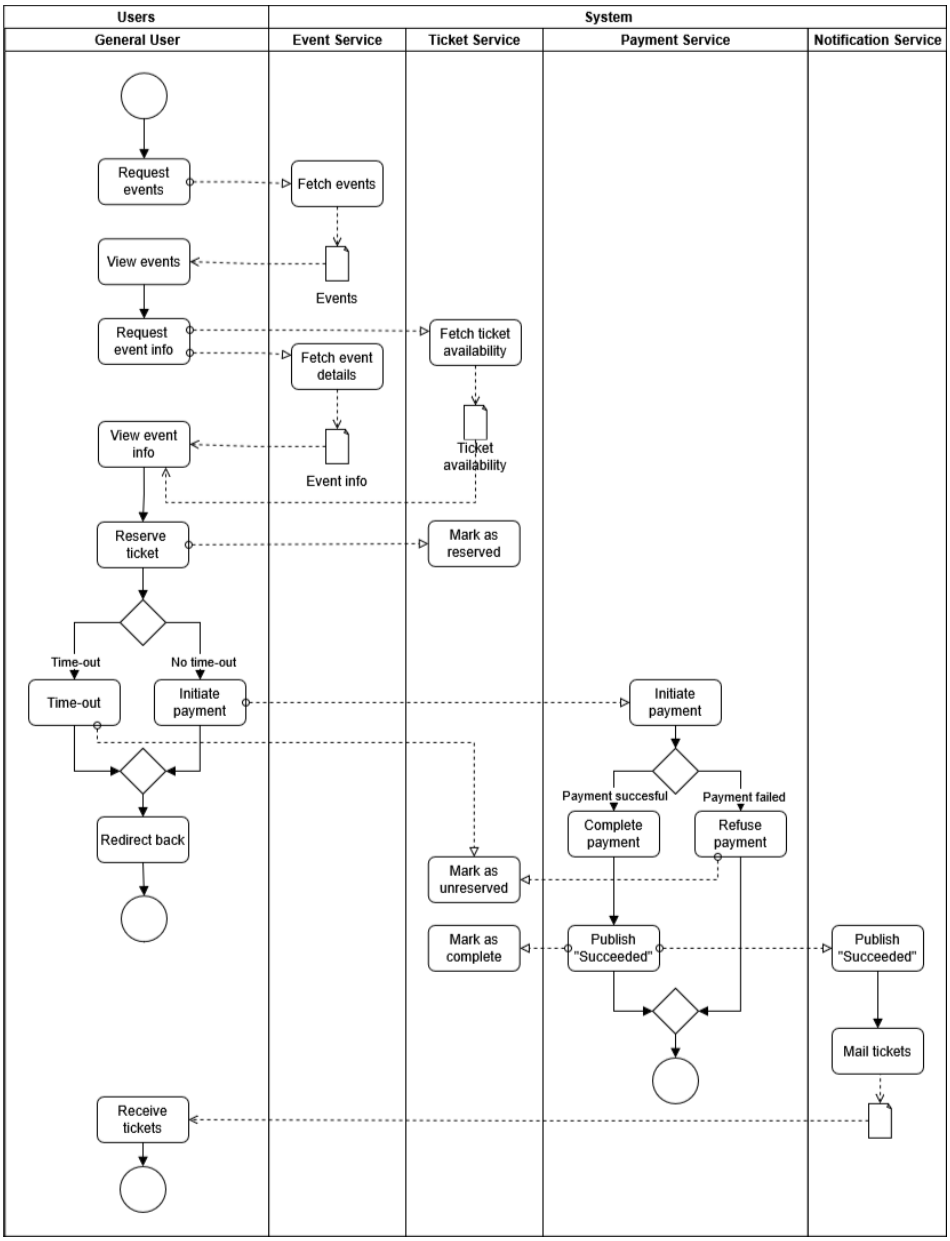

Figure 4: BPMN model of buying a ticket.

5 Services and Endpoints

For milestone 2, the dummy functionality of all endpoints can be found at https://github.com/XantiLizanzu/eventhub. To run the project yourself please follow the steps denoted in the README. All services were implemented as Spring Boot servers with the H2 in memory database and Java Persistence API (JPA). To automatically create the OpenAPI specifications and to serve the Swagger UI, SpringDoc was also added as a dependency. For now, we used Docker Compose to run the services. For the final deliverable, this will all be transformed into the Kubernetes cluster as described in the architecture in Figure 1.

As mentioned, the OpenAPI specifications were generated by the code, rather than developing a separate specification. By generating the docs from code, consistency can be ensured and less manual effort is required. With annotations such as `@Tag` for different controllers and `@Operation` for endpoints, documentation is still rich.

The Events Service consists of the 5 simple CRUD operations (see Figure 5).

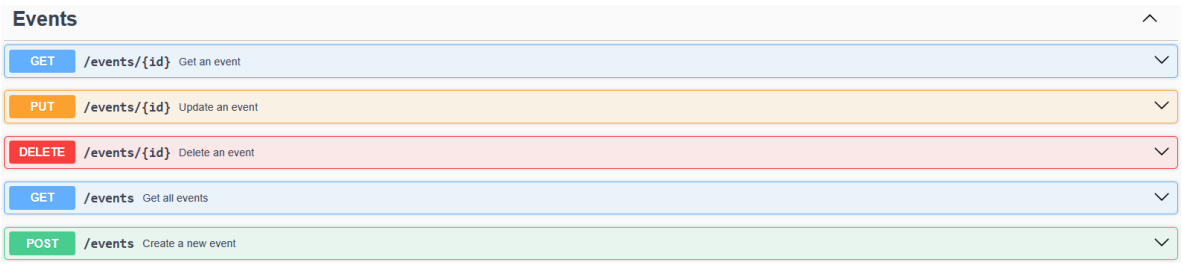

Figure 5: Events Service Swagger UI.

The Payments Service consists of a single endpoint to initialize a payment for a ticket (see Figure 6).

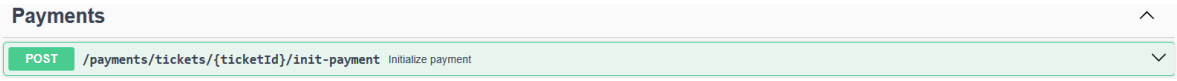

Figure 6: Payments Service Swagger UI.

The Ticketing Service has logic to reserve a ticket, unreserve a ticket, and complete the reservation of a ticket (see Figure 7), according to the BPMN model in Figure 4.

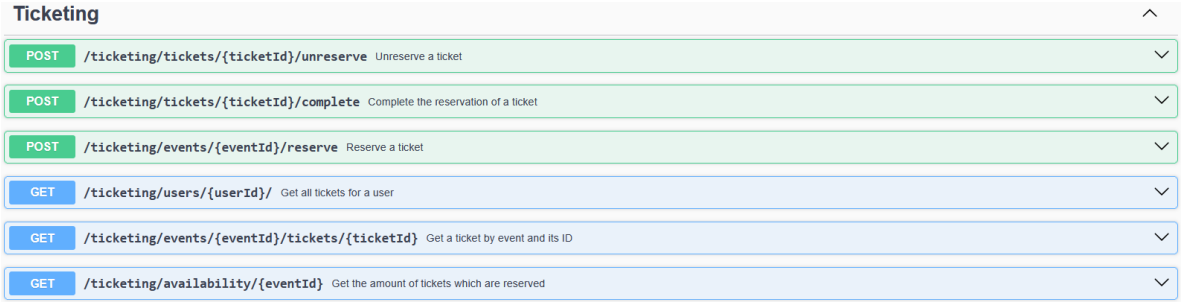

Figure 7: Ticketing Service Swagger UI.

The Users Service consists of two parts, the organizers and the users section. An organizer is a superclass of user, which has an additional name field which is shown in all frontends to communicate who is organizing an event. The organizers section consists of the 5 simple CRUD operations, and the users section consists of the endpoints to registrate, log in and log out (see Figure 8).

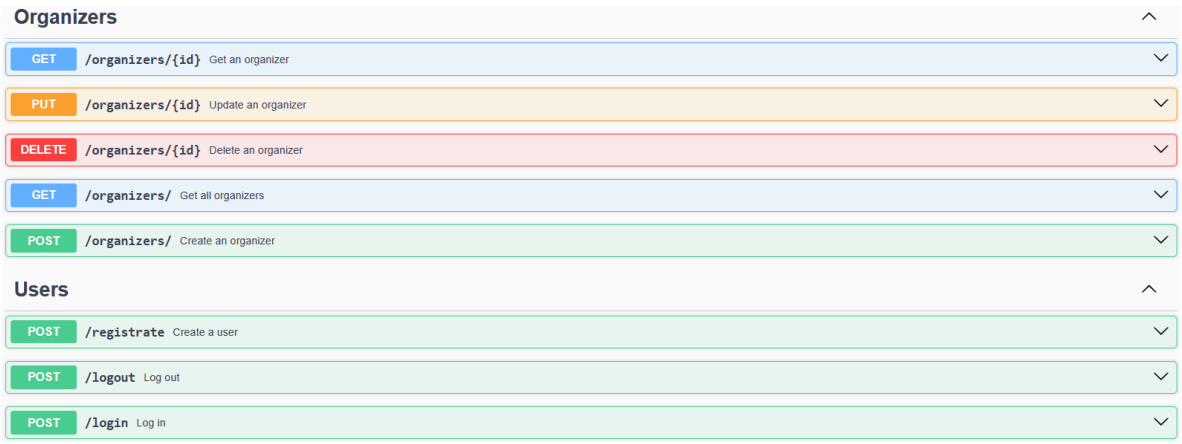

Figure 8: Users Service Swagger UI.

6 Service interactions

In this section, we discuss the currently existing service interactions we implemented. This list is non-exhaustive with regard to the service interactions the production application could support, but is sufficient to showcase the different possibilities with our architecture.

6.1 Synchronous service interactions

6.1.1 Get available tickets

The current implementation of a synchronous service interaction is between the event and ticketing service. The event service has a function getAvailableTickets(eventId) that gets the amount of available tickets from the ticketing service via /ticketing/availability/eventId. We used RestTemplate because it is a simple synchronous HTTP client built into Spring Boot. In Figure 4, the interaction is still drawn from the user to the ticket service. However, with this implementation, that is no longer necessary; The user would only need to directly interact with the event service. The current implementation allows the two services to interact one-to-one after docker compose up is called, since Kubernetes is not configured yet.

6.1.2 Subscribe user to event update notifications

Event updates will send notifications to subscribed users. Users can be subscribed to event updates using POST /notifications/subscriptions/events. This requires checking whether an event exists first, which is done via a synchronous REST call to the events service. For this, we defined an `EventClient` class as a service (`org.springframework.stereotype.Service`) and added an `eventExists()` method that calls the events service to get the event.

6.2 Asynchronous service interactions

6.2.1 Send event update notifications to subscribed users

We developed an interaction between the events service and the notifications service, for which the sequence diagram can be found in Figure 9. This view does not depict the internal routing between Kubernetes components, since this is already depicted in Figure 2. This interaction is modeled in BPMN in Figure 3. The goal of this interaction in a real-world scenario would be informing subscribed/interested users of event updates, such as artist announcements or ticket availability. The diagram depicts the scenario where a user of the type organizer updates an event using a PUT request to the events service. In case the update to the event succeeds, the event Java object is serialized to a string using the Jackson `ObjectMapper`. The string is then inserted into the RabbitMQ `event.updated` queue[2].

[2]Queue names can be configured using environment variables.

The notification service is subscribed to this queue via a `@RabbitListener` annotated listener method, which gets called when a new message is received. Using the `ObjectMapper`, the message string gets deserialized to an Events object. Then, for every user subscribed to event notifications, a message gets printed to the console. If this succeeds, the message is acknowledged using `channel.basicAck`. In case the listener method errors out, the try-catch block ensures that a negative acknowledgement is sent (`channel.basicNack`), and the message is inserted back into the queue. Were this a real application, the next step would be to actually send the message over email to the recipient users. Next to that, notifications should ideally be put in a per-user message queue first, since currently, a failed notification to one user results in the message being negatively acknowledged for all users.

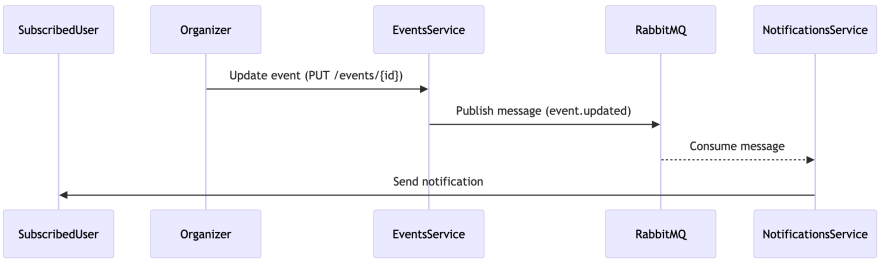

Figure 9: Asynchronous interaction between the events service and notifications service.

6.3 Interdependence issues

The interdependency of these services poses challenges regarding referential integrity and data consistency. Specifically, the deletion of a primary entity (e.g., an event) lacks a defined protocol for managing dependent records, such as notification subscriptions or ticket allocations. While current mitigations are not yet implemented, a proposed architectural refinement involves adopting a soft-delete strategy. This would allow for controlled, cascading deletions across related datasets to maintain historical accuracy.

Additionally, these service interdependencies introduce concurrency risks. Notably, race conditions during high-frequency transactions (such as simultaneous ticket purchases) can lead to a state where the number of issued tickets exceeds available inventory. Figure 10 illustrates a race condition occurring during concurrent ticket reservation requests. As shown, both user 1 and 2 receive confirmation of availability because the TicketsService queries the database before any decrement has been committed to the database. Because both requests retrieve an identical count of one remaining ticket, the system proceeds to execute two separate "reserve ticket" operations. This overlapping execution window results in a final state where more tickets are booked than were initially available. Such an inconsistency highlights the critical need for atomic transactions or distributed locking to maintain data integrity.

7 Validation

We employed a unit testing strategy to validate every controller and service method in the application. For each method, we created focused tests that exercised both the happy path and all relevant edge and failure cases.

For the service layer, we wrote unit tests that targeted each public method in isolation. External dependencies (such as repositories, APIs, and other services) were replaced with mocks or stubs, allowing us to simulate different scenarios and verify that each method returned the correct result or unhappy paths. An example of a service test is

```
void getAllEvents_shouldReturnAllEvents() {
    // Arrange
    Event event2 = new Event("Event 2", "Description 2",
        LocalDateTime.now(), "Location 2", 5);
    when(eventRepository.findAll()).thenReturn(Arrays.asList(
        testEvent, event2));
```

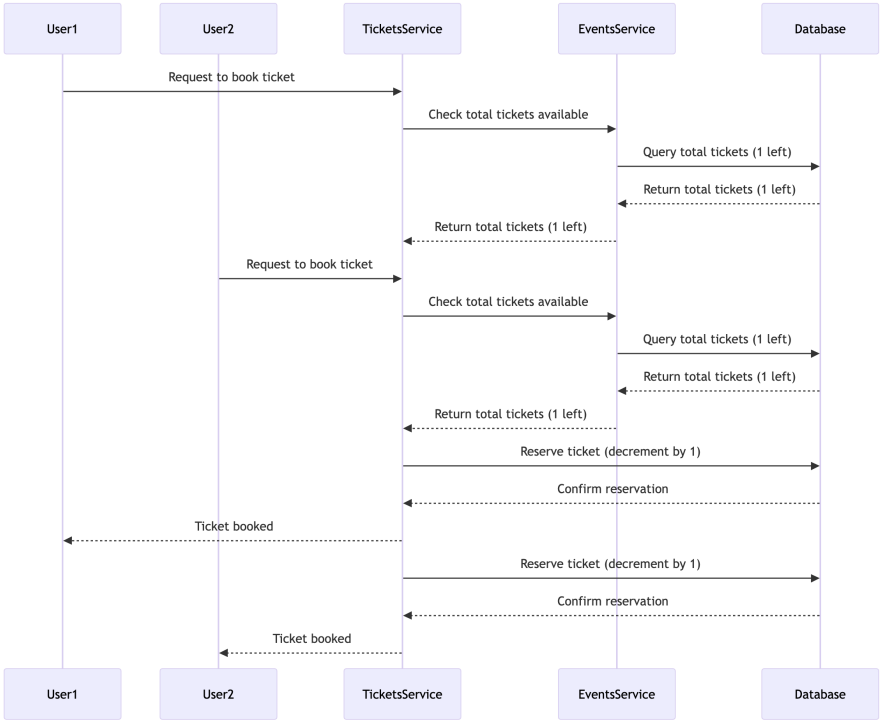

Figure 10: Sequence diagram of a race condition that can occur when two users book a ticket in fast succession.

```
    // Act
    List<Event> result = eventService.getAllEvents();

    // Assert
    assertEquals(2, result.size());
    verify(eventRepository, times(1)).findAll();
}
```

For the controller layer, we tested each endpoint method as an independent unit while mocking the underlying services. These tests ensure that the controllers calls the appropriate service methods and that responses were correct in terms of status codes, response bodies, and error handling. An example of a controller test is:

```
void getEventById_shouldReturnNotFoundWhenNotFound() {
    // Arrange
    when(eventService.getEventById(anyLong()))
        .thenReturn(Optional.empty());

    // Act
    ResponseEntity<EventResponseDTO> result = eventController
        .getEventById(1L);

    // Assert
    assertTrue(result.getStatusCode().is4xxClientError());
    assertNull(result.getBody());
    verify(eventService, times(1)).getEventById(1L);
}
```

Across both controllers and services, we structured tests with clear Arrange–Act–Assert phases and meaningful test names. This approach gave us confidence that our core application logic behaves correctly.

References

[1] Jean-Paul Carmona. *Archimate and Kubernetes concepts mapping*. [Online; accessed 27. Mar. 2026]. Sept. 2025. URL: https://community.opengroup.org/archimate-community/home/-/issues/102.

[2] Sergey Startcev. *What is the most correct way to show microservice infrastructure on the technical level (for example, what objects must be used for docker, docker image, docker host, host machine and etc.) ?* [Online; accessed 27. Mar. 2026]. Oct. 2020. URL: https://community.opengroup.org/archimate-community/home/-/issues/10.

[3] Chris Richardson. *Pattern: Database per service*. [Online; accessed 12. Mar. 2026]. URL: https://microservices.io/patterns/data/database-per-service.html.

A Architecture

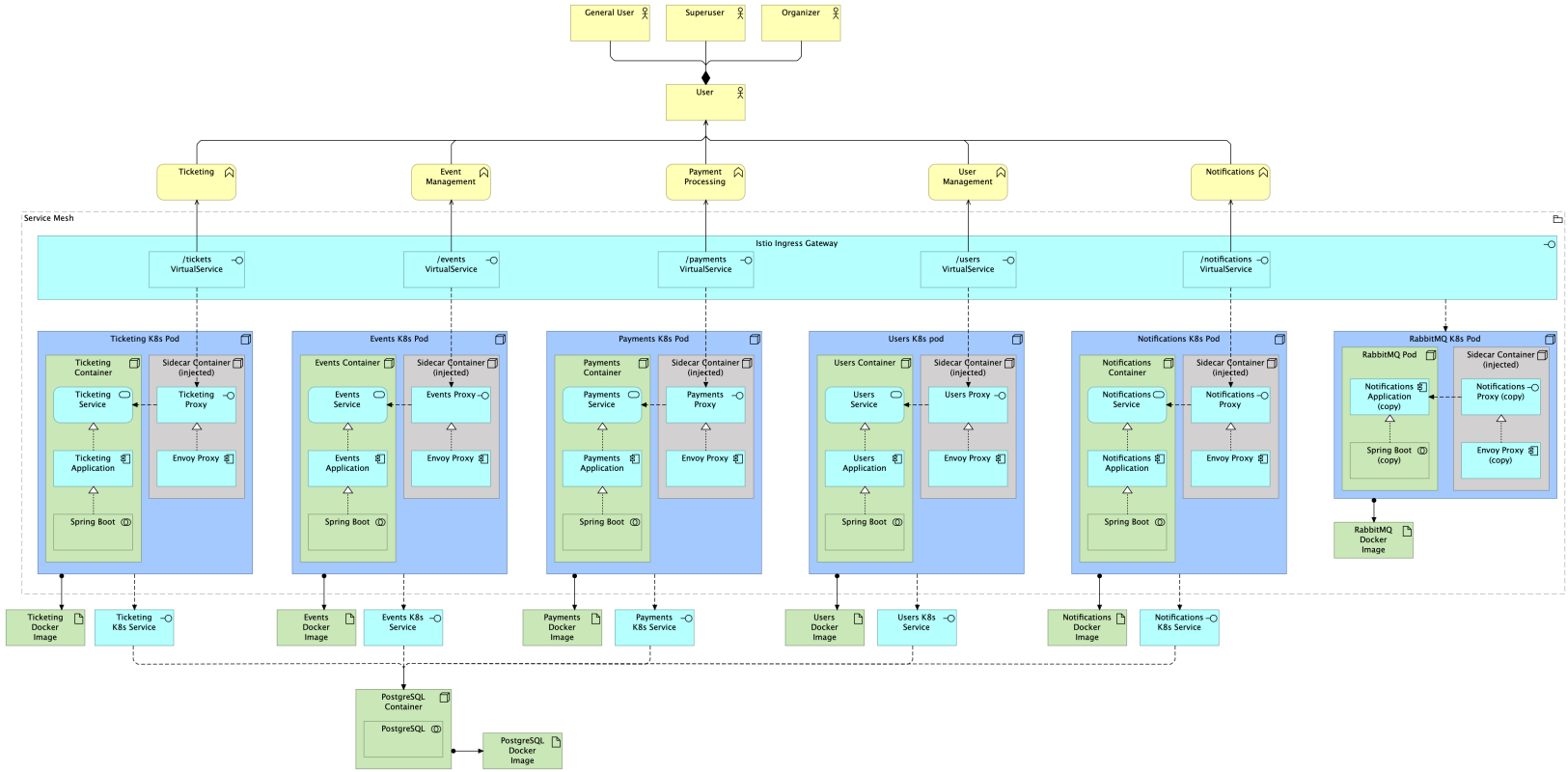

Figure A.1: Enterprise architecture of EventHub.